Procedimiento para Armado de Backlog

Guía práctica para la definición y organización del proyecto

Sobre este documento

Este documento es una **guía práctica** para que tu equipo pueda definir y organizar el backlog de su proyecto. No es un manual teórico: es un procedimiento paso a paso que van a ir completando a lo largo de las primeras clases del curso.

Muchos de los conceptos que aparecen acá (diagramas UML, requerimientos, reglas de negocio) ya los vieron en materias como Análisis y Diseño de Sistemas. La diferencia es que acá los van a aplicar en un **proyecto real de software** que van a construir durante todo el año.

La idea es que cada equipo busque un **cliente real** a quien puedan entrevistar y generarle valor con la aplicación a desarrollar. El proceso de este documento refleja esa dinámica: van a relevar necesidades con el cliente, validar su entendimiento, y presentarle avances. Esto hace que la experiencia sea lo más cercana posible a un proyecto profesional.

Qué tienen que hacer

Para la Entrega 1 del TP grupal, el equipo debe completar como mínimo:

- **Fase 1:** Alcance definido, glosario de términos, diagrama contextual.
- **Fase 2:** Modelo de dominio (diagrama de clases simplificado), reglas de negocio principales.
- **Fase 3:** Épicas, features e historias de usuario priorizadas y estimadas.
- **Wireframes** de al menos 3 pantallas principales (se trabajan en la clase de UX/UI).
- **Validación con el cliente**: haber presentado el backlog al cliente y documentado su feedback.

Todo debe estar en el repositorio del equipo, dentro de una carpeta llamada **Documentación**, organizado por fases.

Índice

Sobre este documento..... 1
Qué tienen que hacer..... 1
Índice..... 2
Fase 1: Preparación y Análisis Inicial..... 3
1.1 Definir el alcance, módulos y límites..... 3
1.2 Glosario de términos..... 4
Fase 2: Modelado de Dominio..... 5
2.1 Modelado de Dominio..... 5
2.2 Requisitos funcionales..... 6
2.3 Reglas de negocio..... 6
2.4 Diagramas complementarios..... 7
Diagramas de estado..... 7
Diagramas de actividad..... 8
Fase 3: Elaboración del Product Backlog..... 9
Conceptos clave..... 9
3.1 Definir épicas y features..... 9
3.2 Escribir Historias de Usuario por Feature..... 10
3.3 División de historias..... 12
3.4 Calendario de planificación..... 13
Fase 4: Validación y Refinamiento..... 15
4.1 Feedback y validación del cliente..... 15
4.2 Refinamiento iterativo..... 15
Fase 5: Seguimiento y Mejora Continua..... 16
5.1 Métricas y KPIs..... 16
5.2 Retrospectivas de proceso..... 16
5.3 Adaptación del proceso..... 16
Anexos..... 17
Anexo A: Plantilla de Historia de Usuario..... 17
Anexo B: Checklist de Calidad para Historias de Usuario..... 17

Fase 1: Preparación y Análisis Inicial

El objetivo de esta fase es entender qué van a construir, para quién, y cuáles son los límites del sistema. El insumo principal es la conversación con el cliente: ¿qué problema tiene? ¿qué necesita?

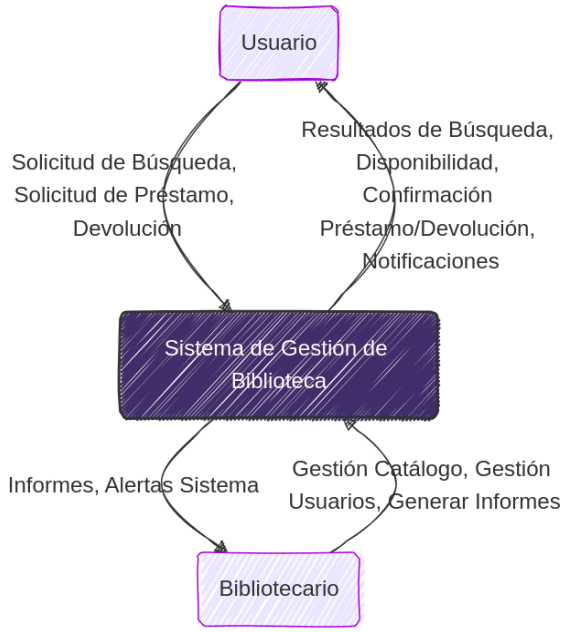
1.1 Definir el alcance, módulos y límites

Reunirse con el equipo y, si es posible, con el cliente las veces necesarias. Responder estas preguntas:

- **¿Qué problema resuelve nuestra aplicación?**
- **¿Quiénes son los usuarios?** (roles: admin, usuario común, etc.)
- **¿Qué puede hacer el sistema y qué NO puede hacer?** (límites claros)
- **¿Con qué sistemas externos interactúa?** (APIs, servicios de pago, email, etc.)
- Si hay dudas o zonas grises, **documentar preguntas para el cliente** y resolverlas en la próxima reunión.

Documentar esto en un **diagrama contextual**: un diagrama simple que muestre el sistema en el centro y los actores/sistemas externos alrededor, con flechas indicando qué interacciones existen.

Ejemplos de Diagrama Contextual:





1.2 Glosario de términos

Crear una tabla con los términos clave del dominio. Esto evita ambigüedades dentro del equipo y sirve como referencia durante todo el desarrollo. Validar los términos con el cliente para asegurar que están usando el mismo vocabulario.

Identificar los sustantivos (posibles entidades) y verbos (posibles acciones/relaciones):

- Crear una lista compartida de estos términos
- Agregar una definición
- Sinónimos: Agrupar términos relacionados (Ej: Asociado, Profesional, Miembro → probablemente sean una misma entidad)
- Clasificación por **módulos** funcionales o grandes bloques del sistema

Ejemplo de Glosario:

Término	Definición	Sinónimos	Módulo
Libro	Obra intelectual (título, autor, ISBN), independiente de sus copias físicas.	Título, Obra	Catálogo
Ejemplar	Copia física de un Libro con código de barras único. Es lo que se presta.	Ítem, Copia	Catálogo
Usuario	Persona registrada con derechos para interactuar con el sistema.	Miembro, Socio	Usuarios
Préstamo	Registro de un Usuario tomando posesión temporal de un Ejemplar.	Retiro	Préstamos
Reserva	Solicitud de prioridad para cuando un Ejemplar no disponible se libere.	Hold	Reservas

Fase 2: Modelado de Dominio

El objetivo es definir la estructura de datos del sistema y las reglas que lo gobiernan.

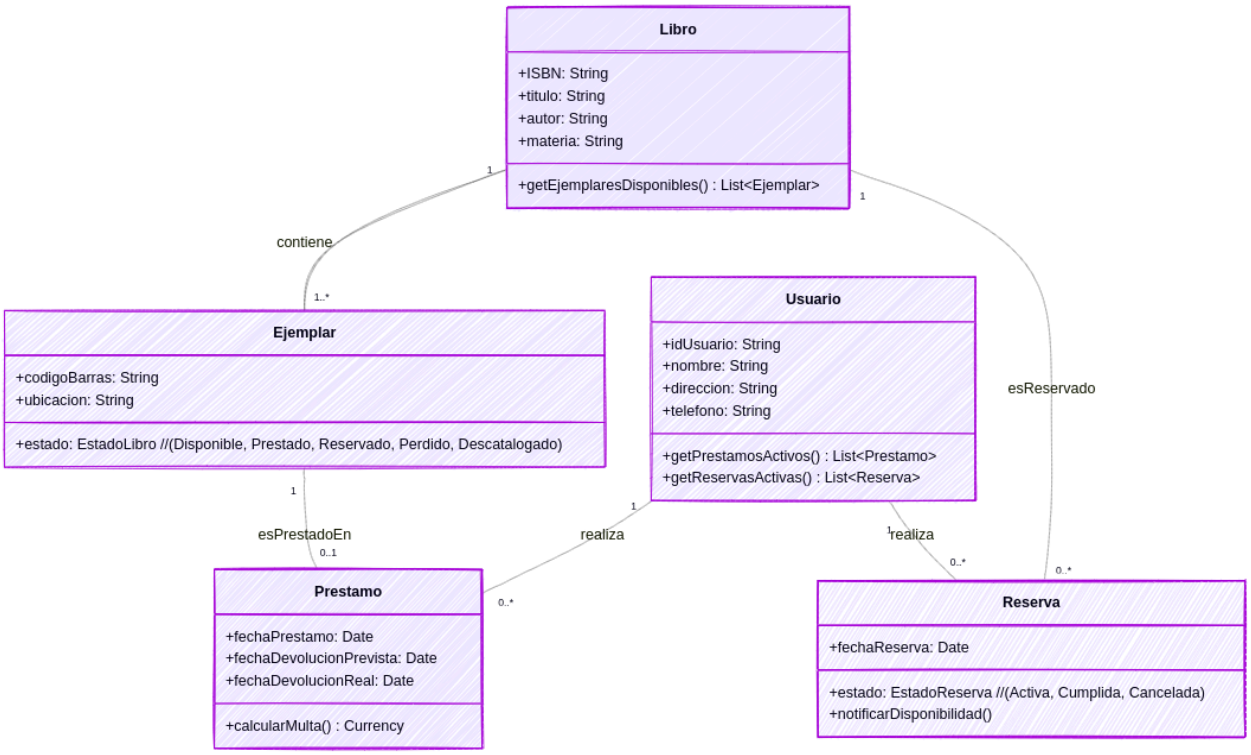
2.1 Modelado de Dominio

Crear un **diagrama de clases UML simplificado** que muestre las entidades principales, sus atributos clave y las relaciones entre ellas. No hace falta que sea exhaustivo: enfocarse en lo que van a implementar.

El diagrama debe incluir:

- Entidades principales con sus atributos más importantes
- Relaciones entre entidades (1:1, 1:N, N:M) con cardinalidad
- Métodos o acciones clave (opcional en esta etapa)

Ejemplo de Diagrama de Clases UML Simplificado:



2.2 Requisitos funcionales

Documentar los **requisitos funcionales** principales: qué debe hacer el sistema desde la perspectiva del usuario. Usar un ID con prefijo RF. Validar con el cliente que cubren sus necesidades.

Ejemplo de Lista de Requisitos Funcionales:

ID	Requisito	Descripción	Módulo
RF-001	Registro de préstamos	El sistema debe registrar los préstamos y detectar devoluciones retrasadas.	Préstamos
RF-002	Gestión de usuarios	El sistema debe almacenar datos personales de los usuarios.	Usuarios
RF-003	Búsqueda	El sistema debe permitir buscar libros por título, autor o categoría.	Catálogo
RF-004	Seguridad de acceso	El sistema debe restringir el acceso a información sensible según el rol.	Seguridad

2.3 Reglas de negocio

Documentar las reglas que definen bajo qué condiciones opera el sistema. Cada regla con un ID (prefijo RN) para referenciarla desde las historias de usuario. Validar estas reglas con el cliente para confirmar que el entendimiento es correcto.

La diferencia: un **requisito funcional (RF)** describe **qué hace** el sistema. Una r**egla de negocio (RN)** describe **bajo qué condiciones** lo hace.

¿Cuántas? Entre 8 y 15 reglas es un rango razonable para un proyecto del curso.

Ejemplo de Lista de Reglas de Negocio:

ID	Regla	Descripción
RN-001	Límite de préstamos	Cada usuario puede tener máximo 5 ítems prestados simultáneamente.
RN-002	Duración del préstamo	El período estándar es de 14 días naturales.
RN-003	Multa por retraso	Se aplica $100 por día de retraso por cada ítem.
RN-004	Bloqueo por multas	Un usuario con multas pendientes no puede hacer nuevos préstamos.
RN-005	Renovaciones	Un préstamo puede renovarse máximo 2 veces, salvo que haya reserva activa.

2.4 Diagramas complementarios

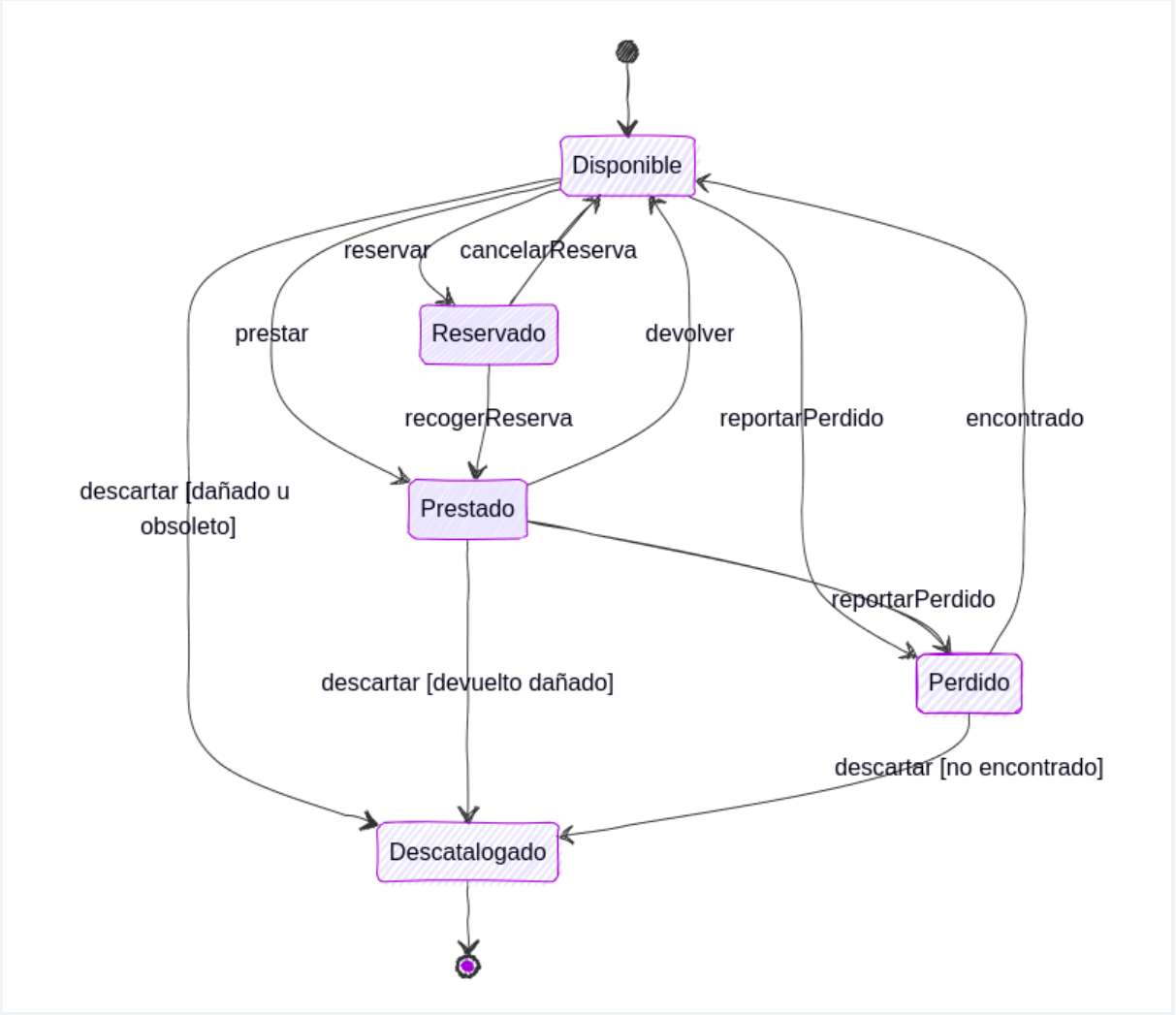
Hay dos tipos de diagramas útiles para documentar los flujos más complejos. No es necesario hacerlos para todos los flujos, solo para los que sean realmente ameriten.

Diagramas de estado

Muestra los estados posibles de una entidad y qué eventos provocan las transiciones.

- **Cuándo usarlo**: Para entidades con ciclo de vida complejo (ej: Ejemplar → Disponible/Prestado/Reservado/Perdido).
- **Cuándo NO usarlo**: Para entidades sin transiciones de estado.

Ejemplo de Diagrama de Estado para "Ejemplar":

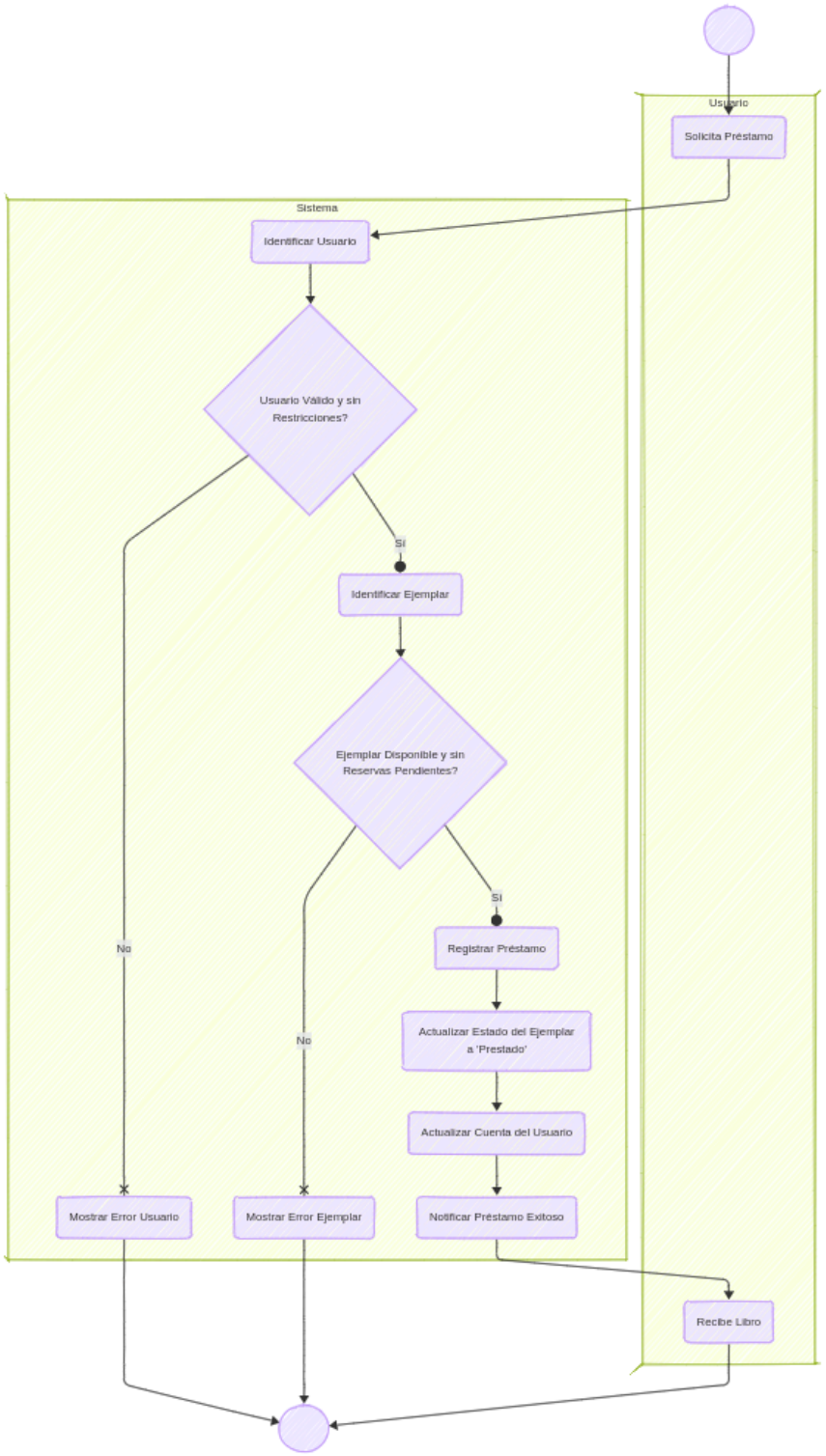


Diagramas de actividad

Muestra el flujo paso a paso de un proceso. Útil para flujos con decisiones.

- **Cuándo usarlo**: Para los 2-3 procesos más complejos (ej: proceso de préstamo, proceso de pago).
- **Cuándo NO usarlo**: Para flujos simples tipo CRUD que son autoexplicativos.

Ejemplo de Diagrama de Actividad para "Préstamo de Libro":



Fase 3: Elaboración del Product Backlog

Esta es la fase central. Acá transforman todo el análisis anterior en un backlog organizado y priorizado que van a usar durante todo el año.

Conceptos clave

Concepto	Qué es	Ejemplo (Biblioteca)
Épica	Un bloque grande de funcionalidad.	Gestión de Préstamos
Feature	Una funcionalidad concreta dentro de una épica.	Proceso de Devolución
Historia de Usuario	Una descripción breve y sencilla de una funcionalidad, escrita desde la perspectiva del usuario final.	Como usuario, quiero devolver un libro escaneando su código.

La relación es:

Épica → tiene varias Features → cada Feature tiene varias Historias de Usuario.

3.1 Definir épicas y features

- Crear las **épicas** partiendo de los módulos principales del sistema
- Descomponer cada épica en **features** más manejables (Asegurar que cada feature genera valor para el usuario)
- Priorizar según importancia para el negocio
    - Método MoSCoW: Must have (Debe), Should (Debería), Could (Podría) y Won't (No debe)
    - Método RICE: Reach (Alcance), Impact (Impacto), Confidence (Confianza), Effort (Esfuerzo)
    - Otros métodos para fijar prioridades
- Estimación inicial: T-shirt sizing (XS, S, M, L, XL) o Planning Poker (1, 2, 3, 5, 8, 13, 21 y 34)
    - Estimar el tamaño relativo de cada feature
    - Identificar features que requieren investigación técnica (spikes)
    - Documentar supuestos y riesgos potenciales

Ejemplo:

Épica	Feature	Prioridad	Tamaño
EP-01: Gestión del Catálogo	FT-001: Búsqueda y navegación	Must	M
EP-01: Gestión del Catálogo	FT-002: Alta/baja de libros	Must	M
EP-02: Gestión de Préstamos	FT-003: Préstamo de libro	Must	L
EP-02: Gestión de Préstamos	FT-004: Devolución de préstamo	Must	S
EP-02: Gestión de Préstamos	FT-005: Renovación de préstamo	Should	S
EP-03: Gestión de Reservas	FT-006: Hacer una reserva	Should	S
EP-04: Multas y Pagos	FT-007: Cálculo automático de multas	Must	M
EP-05: Autenticación	FT-008: Login / Registro	Must	M
EP-06: Notificaciones	FT-009: Notificar Disponibilidad Reserva	Should	S
EP-06: Notificaciones	FT-010: Recordatorios Vencimiento	Should	S
EP-07: Migración de Datos	FT-011: Migración Datos	Must	XL

3.2 Escribir Historias de Usuario por Feature

Cada feature se transforma en una o más **historias de usuario**, siguiendo el formato básico:

```
Como [rol], quiero [objetivo] para [beneficio].
```

Las historias se escriben **una sola vez**, desde la perspectiva del usuario, sin separarlas por capa técnica (frontend/backend).

Se acompañan con:

- Asignar prioridad inicial: MoSCoW, RICE u otro método
- Estimación inicial: T-shirt sizing o Planning Poker
- Criterios de aceptación de alto nivel en formato Given/When/Then:
  - Given (dado/contexto)
  - When (cuándo/evento)
  - Then (entonces/resultado)
- Referencias a requisitos funcionales con su ID
- Referencias a reglas de negocio con su ID

- Notas técnicas si aplica (para desarrolladores)
- Adjuntar wireframes o mockups según sea necesario

Cosas importantes a tener en cuenta:

- Asegurar que cada historia genera valor para el usuario
- Usar una plantilla estandarizada de historias de usuario (ver Anexo A)
- Aplicar la Checklist de Calidad (ver Anexo B) a cada historia

> **¿Por qué no separar historias en Frontend y Backend?**
>
> La historia describe lo que el usuario necesita, no cómo se implementa. La separación por capas se resuelve en la planificación (sección 3.5), donde se define qué parte de cada historia se implementa en cada bloque del curso.

Ejemplo Simple

```
---------------------------------------------------------------------

[HU-001] - Buscar libros por título

Descripción:
Como usuario,
quiero buscar libros por título,
para encontrar rápidamente el que necesito.

Épica: [EP-01] Gestión del Catálogo
Prioridad: Must
Estimación: S

Criterios de Aceptación:
1. Dado que estoy en el catálogo, Cuando escribo un título en el buscador,
   Entonces se muestran los libros que coinciden.
2. Dado que busco un título que no existe, Cuando presiono buscar, Entonces se
   muestra "No se encontraron resultados".

Requisitos Funcionales: [RF-003]

Reglas de Negocio: -

---------------------------------------------------------------------
```

Ejemplo Detallado

```
---------------------------------------------------------------------

[HU-004] - Solicitar préstamo de libro

Descripción:
Como usuario registrado,
```

quiero solicitar el préstamo de un libro disponible,
para llevármelo a casa.

Épica: [EP-02] Gestión de Préstamos
Prioridad: Must
Estimación: L

Criterios de Aceptación:
1. **Dado** que soy un usuario válido con menos del máximo de préstamos, **cuando** solicito un ítem disponible, **entonces** se registra el préstamo, se actualiza el estado y se muestra confirmación.
2. **Dado** que ya alcancé el límite de préstamos (RN-001), **cuando** intento solicitar otro, **entonces** se muestra "Límite de préstamos alcanzado".
3. **Dado** que el ítem no está disponible, **cuando** intento solicitarlo, **entonces** se muestra "Ítem no disponible".
4. **Dado que tengo multas pendientes (RN-004), cuando** intento un préstamo, **entonces** se muestra "Tiene multas pendientes".

Notas Técnicas:
- Actualizar estado del ejemplar en la DB. Considerar transacciones para atomicidad.

Requisitos Funcionales:
- [RF-001] Registro de préstamos

Reglas de Negocio:
- [RN-001] Límite de préstamos
- [RN-002] Duración del préstamo
- [RN-004] Bloqueo por multas

Archivos Adjuntos:
- “HU-004 - Wireframe - Formulario Solicitud de Préstamo.pdf”
- “HU-004 - Wireframe - Modal de mensaje Error.pdf”

3.3 División de historias

Si una historia es muy grande (estimada como L o XL), dividirla en partes más chicas (1 a 3 días de trabajo). Verificar los criterios INVEST:

- Independent (Independiente)
- Negotiable (Negociable)
- Valuable (Valiosa)
- Estimable (Estimable)
- Small (Pequeña)
- Testable (Comprobable)

En este enlace hay una infografía útil sobre cómo ejecutar la división:
UTN DS26 - C04 - Story-Splitting-Flowchart-ES.pdf

No olvidar:

- Documentar dependencias entre historias
- Aplicar la Checklist de Calidad (ver Anexo B) a cada historia

3.4 Calendario de planificación

Con el backlog armado, podemos distribuir las historias en un calendario que refleje la estructura del curso. En la materia, el desarrollo se da en etapas secuenciales:

Bloques	Período	Qué se implementa
Bloque 2: Frontend	Mayo a Julio	La interfaz de usuario completa, con datos mock. Formularios, navegación, validaciones front.
Bloque 3: Backend	Julio a Septiembre	La API, base de datos, autenticación. Todo el servidor, probado con Postman.
Bloque 4: Integración	Septiembre a Noviembre	Conectar front con back real. Autorización, deploy, testing, ajustes finales.

Las historias se escriben una sola vez pero se implementan en fases. En la herramienta de gestión, cada historia puede tener **tareas** asociadas que indiquen en qué bloque se trabaja cada parte:

Ejemplo: HU-004 — Solicitar préstamo

- **Bloque Front**: Pantalla de préstamo, formulario, validaciones, estado local, datos mock.
- **Bloque Back**: Endpoint POST /api/prestamos, lógica de negocio, validaciones Zod, Prisma.
- **Integración**: Conectar el formulario al endpoint real, manejar errores, auth.

Lo que deben hacer es planificar los primeros 2-3 sprints con historias priorizadas (objetivos definidos)

Ejemplo de Calendario** (Basado en la lista 3.1)**:

FRONTEND		
SPRINT (2 semanas)	**Épica**	**Feature**
FR-1	EP-01: Gestión del Catálogo	FT-002: Alta/baja de libro
	EP-01: Gestión del Catálogo	FT-001: Búsqueda y navegación
	EP-02: Gestión de Préstamo	FT-003: Préstamo de libro

	EP-02: Gestión de Préstamo	FT-004: Devolución de préstamo
FR-2	EP-05: Autenticación	FT-008: Login / Registro
	EP-04: Multas y Pagos	FT-007: Cálculo automático de multas
	EP-02: Gestión de Préstamos	FT-005: Renovación de préstamo
	EP-02: Gestión de Préstamos	FT-006: Hacer una reserva

BACKEND		
SPRINT (2 semanas)	**Épica**	**Feature**
BK-1	EP-01: Gestión del Catálogo	FT-002: Alta/baja de libro
	EP-01: Gestión del Catálogo	FT-001: Búsqueda y navegación
	EP-07: Migración de Datos	FT-011: Migración Datos
BK-2	EP-02: Gestión de Préstamo	FT-003: Préstamo de libro
	EP-02: Gestión de Préstamo	FT-004: Devolución de préstamo
	EP-05: Autenticación	FT-008: Login / Registro
BK-3	EP-04: Multas y Pagos	FT-007: Cálculo automático de multas
	EP-02: Gestión de Préstamos	FT-005: Renovación de préstamo
	EP-02: Gestión de Préstamos	FT-006: Hacer una reserva

Fase 4: Validación y Refinamiento

4.1 Feedback y validación del cliente

Una vez que tienen el backlog armado y priorizado, **presentárselo al cliente**. Esta reunión es clave para confirmar que entendieron bien lo que necesita y que las prioridades están alineadas. Es normal que surjan ajustes: nuevas historias, cambios de prioridad, o aclaraciones sobre reglas de negocio. Incorporar ese feedback antes de empezar a desarrollar.

- Presentar el backlog priorizado al cliente
- Validar el entendimiento y las prioridades
- Incorporar feedback en tiempo real
- Documentar cambios o aclaraciones

4.2 Refinamiento iterativo

La validación del punto anterior no es un evento único: a lo largo del año, en cada entrega parcial, deberían mostrar avances al cliente y recibir feedback. El backlog es un documento vivo que evoluciona con cada iteración.

- Programar sesiones de refinamiento periódicas en el equipo (bi-semanales)
- Revisar y ajustar las historias próximas en la priorización
- Actualizar el backlog basado en nuevos conocimientos o cambios del mercado

Fase 5: Seguimiento y Mejora Continua

5.1 Métricas y KPIs

- Definir métricas clave para medir el éxito del proyecto:
  - Velocidad del equipo
  - Calidad del código (bugs, deuda técnica)
  - Satisfacción del cliente
  - Cobertura de pruebas
- Establecer un dashboard para visualizar estas métricas

5.2 Retrospectivas de proceso

- Realizar retrospectivas específicas sobre el proceso de creación del backlog
- Identificar áreas de mejora para futuros proyectos
- Documentar lecciones aprendidas

5.3 Adaptación del proceso

- Ajustar este procedimiento según las lecciones aprendidas
- Mantener versiones del proceso para capturar mejoras
- Compartir mejores prácticas con otros equipos

Anexos

Anexo A: Plantilla de Historia de Usuario

```
---------------------------------------------------------------------

[ID] - Título descriptivo

Descripción:
Como [rol],
quiero [objetivo]
para [beneficio].

Épica: [ID EP-###]
Prioridad: [valor prioridad]
Estimación: [valor estimado]

Criterios de Aceptación:
1. Dado [contexto], Cuando [acción], Entonces [resultado].
2. Dado [contexto], Cuando [acción], Entonces [resultado].
...

Notas Técnicas:
- [Consideraciones de implementación]
- [Referencias a APIs/servicios]
- [Tecnologías a utilizar]

Requisitos Funcionales:
- [RF-001] [Descripción breve]
- [RF-002] [Descripción breve]

Reglas de Negocio:
- [RN-001] [Descripción breve]
- [RN-002] [Descripción breve]

Archivos Adjuntos:
- [Enlaces a mockups, diagramas, etc.]

---------------------------------------------------------------------
```

Anexo B: Checklist de Calidad para Historias de Usuario

Antes de dar por terminada una historia de usuario, verificar:

#	Criterio	✅
1	Sigue el formato "Como [rol], quiero [objetivo] para [beneficio]"	☐
2	Tiene criterios de aceptación claros (Given/When/Then)	☐
3	Es lo suficientemente pequeña (1-3 días de trabajo)	☐
4	Genera valor para el usuario	☐
5	Es testeable (se puede verificar que funciona)	☐
6	Tiene un título descriptivo con ID (ej: HU-001)	☐
7	Referencia las reglas de negocio y requisitos que aplican	☐
8	No duplica otra historia existente	☐

9	Si tiene wireframes asociados, están referenciados	☐